◆ HOKEKA

Edge Engine

On-Premise Installation & Prerequisites Guide

DEPLOYMENT GUIDE | v0.1 | CONFIDENTIAL

Document: Hokeka Edge Engine — Installation Guide

Audience: PSP infrastructure / DevOps / security teams

Scope: installing the on-premise transaction-validation engine on any Linux distribution, Windows Server, or container platform.

Contents

> **About the illustrations in this guide**
> The Edge Engine installs and runs as a headless service (CLI / container / systemd) — there is no graphical installer. The panels styled as terminal windows below are faithful command-and-output walkthroughs of each step, not screenshots of a GUI. Copy the commands as shown.

1 What you are installing

The **Hokeka Edge Engine** is the on-premise component of the Hokeka platform. It runs inside the PSP's own network and validates transactions **locally**, so **customer and transaction data never leave the PSP's premises**. It consists of three co-located pieces delivered as one package:

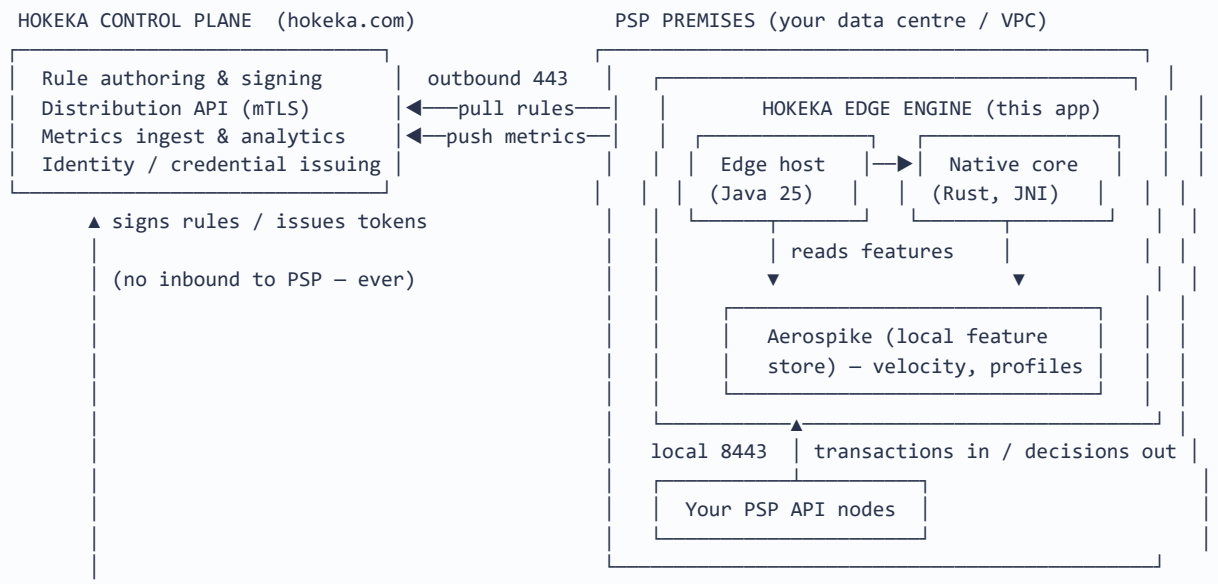
Component	Role	Technology
Edge host	Transaction API for your PSP nodes, offline auth, rule polling, metrics shipping	Java 25 (Spring Boot, virtual threads)
Native core	The rule-evaluation kernel (fast, CPU-bound)	Rust (loaded by the host via JNI: `libedge_engine.so`)
Feature store	Velocity counters, profiles, device/IP reputation	Aerospike (local)

The engine talks to the **Hokeka control plane** over the public internet using **outbound-only** connections for two purposes only:

- **Pull rule bundles** — encrypted and signed (HSE-1 under TLS); rules are authored centrally, never at the edge.
- **Push aggregate metrics** — counts, decisions and latencies only. **No PAN, customer, amount or transaction data is ever sent.**

> **Key principle**
> Rules flow down, aggregate metrics flow up, raw data stays put. No inbound connection from Hokeka to your network is ever required.

2 Architecture at a glance



> **Trust boundary**

> Everything inside the PSP box is yours and stays yours. The only bytes that cross the boundary are signed/encrypted rule bundles (down) and PII-free aggregate metrics (up).

3 Prerequisites

3.1 Hardware sizing

Sizing is driven by sustained transactions-per-second (TPS). The figures below are **per edge instance**; scale horizontally by adding instances behind your load balancer. Aerospike memory holds the primary index and hot feature data — provision RAM accordingly.

Tier	Sustained TPS	vCPU	RAM	Storage	Notes
Pilot / Small	≤ 200	4	16 GB	100 GB SSD	Aerospike co-located; single node
Medium	200 – 1,000	8	32 GB	250 GB NVMe	Co-located or 1 dedicated Aerospike node
Large	1,000 – 5,000	16	64 GB	500 GB NVMe	Dedicated Aerospike node recommended
X-Large	5,000+	32	128 GB	1 TB NVMe	Aerospike cluster; 2+ edge instances

> **Recommendation**
> Prefer NVMe/SSD over spinning disk for Aerospike, enable at least 2 edge instances in production for high availability, and reserve ~25% CPU headroom for burst traffic.

3.2 Operating system support

The engine is 64-bit only and runs on **any modern OS**. Linux is recommended for production.

Platform	Supported versions	Status
Ubuntu / Debian	Ubuntu 22.04 & 24.04 LTS · Debian 12	RECOMMENDED
RHEL / Rocky / AlmaLinux	9.x (and 8.x)	RECOMMENDED
SUSE Linux Enterprise / openSUSE	15 SP5+	SUPPORTED
Amazon Linux / Oracle Linux	2023 / 9	SUPPORTED
Any other glibc Linux distro	glibc ≥ 2.31, kernel ≥ 5.10	SUPPORTED (tarball)
Windows Server	2019 / 2022	SUPPORTED
Containers	Docker 24+ / Podman 4+ on any host	RECOMMENDED (distro-agnostic)
Kubernetes	1.27+	SUPPORTED (Helm chart)

> **Distro-agnostic path**
> If your distribution is not listed, use
> **Method A (container)**
> or
> **Method C (tarball)**
> — both run on any 64-bit glibc Linux without a native package.

3.3 Software prerequisites

Requirement	Version	Why
Java runtime	JDK / JRE **25** (LTS)	Virtual threads for concurrency; JDK 25 removes `synchronized` carrier-pinning. Eclipse Temurin recommended. Bundled in the container image.
Aerospike	Community or Enterprise 6.4+	Local low-latency feature store. Bundled/orchestrated by container & package installs.
glibc	≥ 2.31	Runtime for the Rust native core (`libedge_engine.so`).
System libs	ca-certificates, tzdata	TLS trust store, time zones.
Time sync	chrony / systemd-timesyncd	Token expiry & signature validity depend on accurate clocks.

> **Note on Java**
> You do *not* need to hand-compile anything. The Rust core ships pre-built as `libedge_engine.so` inside the package/image; the host loads it at startup. No compiler or build tools are required on the target machine.

3.4 Network prerequisites

Direction	Port / Protocol	Endpoint	Purpose
Outbound	TCP 443 (HTTPS + mTLS)	`edge.hokeka.com`	Pull rule bundles & push aggregate metrics
Inbound (local)	TCP 8443 (HTTPS)	Your PSP API nodes only	Submit transactions, receive decisions
Local	TCP 3000	Aerospike (localhost / private)	Feature store access

> **Firewall-friendly**
> Only
> **one outbound**
> destination on 443 is required. No inbound internet access to the edge is needed — the edge polls Hokeka, Hokeka never connects in.

3.5 Security prerequisites (provided during onboarding)

- **Edge X25519 key pair** — generated on first run; the public key is registered with Hokeka so rule bundles can be sealed for this edge.
- **Pinned control-plane keys** — Hokeka's Ed25519 signing public key (verifies rule bundles & auth tokens) shipped in the config.
- **mTLS client certificate** — issued per edge at provisioning; identifies this edge to the control plane.
- **PSP / edge identifiers** — `pspId` and `edgeId` assigned by Hokeka.

4 Choosing an installation method

If you…	Use
Want the fastest, most portable, distro-independent install	**Method A — Container** RECOMMENDED
Run a managed fleet on Debian/Ubuntu or RHEL-family hosts	**Method B — Native package**
Run an unlisted/hardened distro or an air-gapped host	**Method C — Tarball**
Must deploy on Windows Server	**Method D — Windows**
Need horizontal scale / self-healing	**Method E — Kubernetes / Helm**

> **Artifacts**
> All images and packages are served from your Hokeka onboarding portal (`registry.hokeka.com` , `packages.hokeka.com`) with credentials issued to your team. Every artifact is signed; verify signatures before install (shown per method).

5 Method A — Container (Docker / Podman) RECOMMENDED

Runs on any 64-bit host with a container runtime. Java, the native core, and Aerospike wiring are bundled.

Step 1 — Authenticate to the Hokeka registry

psp-host: ~/hokeka-edge

```
$ docker login registry.hokeka.com -u psp-acme
Password: ••••••••••••
Login Succeeded
```

Step 2 — Pull & verify the image

psp-host: ~/hokeka-edge

```
$ docker pull registry.hokeka.com/hokeka/edge-engine:0.1.0
0.1.0: Pulling from hokeka/edge-engine
a1b2c3…  Pull complete
Digest: sha256:9f4e…c2   Status: Downloaded newer image
$ cosign verify --key hokeka.pub registry.hokeka.com/hokeka/edge-engine:0.1.0
Verified OK # signature matches Hokeka's release key
```

Step 3 — Create the config & secrets directory

psp-host: ~/hokeka-edge

```
$ mkdir -p ./config ./secrets ./data
$ cp edge.yaml ./config/           # from onboarding pack (see §11)
$ cp hokeka-cp.pub client.crt client.key ./secrets/
```

Step 4 — Run (Compose brings up the edge + local Aerospike)

```
psp-host: ~/hokeka-edge

$ docker compose up -d
[+] Running 2/2
 ✓ Container hokeka-aerospike   Started
 ✓ Container hokeka-edge        Started
$ docker compose logs -f edge | head
edge | INFO  EdgeEngine  loaded native core libedge_engine.so (rust 1.97)
edge | INFO  RulePoller  pulled bundle v37 (sealed HSE-1) — signature OK, activated
edge | INFO  EdgeHost    listening on https://0.0.0.0:8443  virtual-threads=on
edge | INFO  Health      status=HEALTHY  aerospike=UP  controlPlane=REACHABLE
```

> **Podman**
> Every command above works with `podman` / `podman compose` unchanged — handy for rootless RHEL/Fedora hosts.

6 Method B — Native package

6.1 Debian / Ubuntu (`apt`)

```
root@psp-host: ~

# curl -fsSL https://packages.hokeka.com/gpg | gpg --dearmor -o /usr/share/keyrings/hokeka.gpg
# echo "deb [signed-by=/usr/share/keyrings/hokeka.gpg] https://packages.hokeka.com/apt stable main" \
      > /etc/apt/sources.list.d/hokeka.list
# apt update && apt install -y hokeka-edge-engine
Setting up hokeka-edge-engine (0.1.0) ...
Installing bundled Temurin JRE 25, libedge_engine.so, systemd unit hokeka-edge.service
✓ hokeka-edge-engine installed. Configure /etc/hokeka/edge.yaml, then: systemctl start hokeka-edge
```

6.2 RHEL / Rocky / AlmaLinux / Fedora (`dnf`)

```
root@psp-host: ~

# dnf config-manager --add-repo https://packages.hokeka.com/rpm/hokeka.repo
# dnf install -y hokeka-edge-engine
Importing GPG key 0x9F4E…  Fingerprint: 9F4E 22C1 … (Hokeka Packaging)
Complete!  ✓ hokeka-edge-engine-0.1.0 installed
```

6.3 SUSE / openSUSE (`zypper`)

```
root@psp-host: ~

# zypper addrepo https://packages.hokeka.com/rpm/hokeka.repo
# zypper --gpg-auto-import-keys install hokeka-edge-engine
✓ hokeka-edge-engine installed
```

> **What the package installs**
> `/opt/hokeka/edge` (binaries + `libedge_engine.so` + bundled JRE 25), config at `/etc/hokeka/edge.yaml`, data under `/var/lib/hokeka`, and a `hokeka-edge.service` systemd unit. Aerospike is installed as a dependency or pointed at an existing node.

7 Method C — Generic tarball (any distribution)

For unlisted, minimal, hardened, or air-gapped distributions. Self-contained (bundled JRE + native core); no root package manager required.

psp-host: /opt

```
$ curl -fsSLO https://packages.hokeka.com/tarball/hokeka-edge-0.1.0-linux-x86_64.tar.gz
$ curl -fsSLO https://packages.hokeka.com/tarball/hokeka-edge-0.1.0-linux-x86_64.tar.gz.sig
$ cosign verify-blob --key hokeka.pub --signature hokeka-edge-0.1.0-linux-x86_64.tar.gz.sig \
      hokeka-edge-0.1.0-linux-x86_64.tar.gz
Verified OK
$ sudo tar -xzf hokeka-edge-0.1.0-linux-x86_64.tar.gz -C /opt
$ /opt/hokeka-edge/bin/edge --config /opt/hokeka-edge/config/edge.yaml --check
✓ config valid · native core loadable · aerospike reachable · control-plane reachable
$ /opt/hokeka-edge/bin/edge --config /opt/hokeka-edge/config/edge.yaml &
```

Air-gapped installs

Download the tarball + a signed rule-bundle seed on a connected host, transfer via approved media, and set `controlPlane.mode: offline-seed` in the config. The edge runs on the seeded bundle and buffers metrics until connectivity is restored.

8 Method D — Windows Server

Supported on Windows Server 2019 / 2022. Ships as an MSI that registers a Windows Service and bundles the JRE and `edge_engine.dll`.

```
PS C:\hokeka>

$ Get-AuthenticodeSignature .\hokeka-edge-0.1.0.msi | Select Status
Status : Valid
$ msiexec /i hokeka-edge-0.1.0.msi /qn CONFIG="C:\hokeka\edge.yaml"
$ Start-Service HokekaEdge
$ Get-Service HokekaEdge
Status   Name        DisplayName
Running  HokekaEdge  Hokeka Edge Engine
```

> **Aerospike on Windows**
> Aerospike has no native Windows server build — on Windows hosts run the feature store as a container (Docker Desktop / WSL2) or point the edge at a nearby Linux Aerospike node via `featureStore.host`.

9 Method E — Kubernetes / Helm (horizontal scale)

```
psp-host: ~

$ helm repo add hokeka https://charts.hokeka.com && helm repo update
$ kubectl create secret generic hokeka-edge-secrets \
     --from-file=hokeka-cp.pub --from-file=client.crt --from-file=client.key
$ helm install edge hokeka/edge-engine -n hokeka --create-namespace \
     --set pspId=acme --set edgeId=acme-eu-1 --set replicas=3 \
     --set controlPlane.url=https://edge.hokeka.com
NAME: edge   STATUS: deployed   REVISION: 1
$ kubectl -n hokeka get pods
NAME                    READY   STATUS    RESTARTS
edge-engine-0           1/1     Running   0
edge-engine-1           1/1     Running   0
edge-engine-2           1/1     Running   0
aerospike-0             1/1     Running   0
```

> **Scale**
> Increase `--set replicas` and, for Large/X-Large, run Aerospike as a StatefulSet cluster. All edge pods pull the same signed rule bundle and hot-swap it atomically on change.

10 Local Aerospike feature store

Container and Helm installs provision Aerospike automatically. For native/tarball installs on a dedicated node, install Aerospike and create the `hokeka` namespace:

root@aerospike-node: ~

```
# Debian/Ubuntu example — see Aerospike docs for your distro
# curl -fsSL https://download.aerospike.com/…/aerospike.deb -o aero.deb && apt install -y ./aero.deb
# nano /etc/aerospike/aerospike.conf   # add: namespace hokeka { memory-size 8G  storage-engine device {
file /var/lib/aerospike/hokeka.dat  filesize 200G } }
# systemctl enable --now aerospike
$ asadm -e "info"
Node ●  Namespace hokeka  Objects 0  ●  Cluster size 1  ●  Migrations 0
```

> **Sizing**
> `memory-size` ≈ 25–40% of node RAM (holds index + hot data); `filesize` per the storage tier in §3.1. Feature TTLs keep the store bounded automatically.

11 Configuration reference (`edge.yaml`)

/etc/hokeka/edge.yaml

```yaml
edge:
  pspId: acme                      # assigned by Hokeka
  edgeId: acme-eu-1                 # unique per instance
  listen: "0.0.0.0:8443"           # local transaction API (TLS)

controlPlane:
  url: https://edge.hokeka.com
  pollIntervalSeconds: 15           # how often to check for a new rule bundle
  signingKeyFile: /etc/hokeka/secrets/hokeka-cp.pub   # pinned Ed25519 (verify bundles+tokens)
  mtls: { certFile: …/client.crt, keyFile: …/client.key }

featureStore:
  host: 127.0.0.1
  port: 3000
  namespace: hokeka

runtime:
  virtualThreads: true             # concurrency (Java 25)
  maxInFlight: 30000               # admission control / backpressure
  failClosed: true                 # feature store down ⇒ HOLD, never silent ALLOW

metrics:
  shipIntervalSeconds: 60          # aggregate-only; store-and-forward if offline
```

12 Provisioning the edge

On first start the edge generates its X25519 key pair and prints its public key. Register it with Hokeka to begin receiving sealed rule bundles for your PSP.

psp-host: ~

```
$ hokeka-edge provision --config /etc/hokeka/edge.yaml
Generated edge X25519 key pair (stored: /var/lib/hokeka/edge_x25519.key, 0600)
Edge public key (register at portal.hokeka.com):
  x25519:pub  9d4c8a…e2f1
$ # after registering in the portal, the edge pulls its first bundle automatically:
INFO RulePoller  pulled bundle v37 · HSE-1 signature OK · decrypted · 55 rules · activated
```

What just happened

The edge's public key lets Hokeka **seal** (encrypt) rule bundles that only this edge can open, and pin Hokeka's key lets the edge **verify** every bundle and auth token — all offline, no per-transaction call to Hokeka.

13 Starting, verifying & health checks

root@psp-host: ~

```
# systemctl enable --now hokeka-edge
# systemctl status hokeka-edge --no-pager
● hokeka-edge.service - Hokeka Edge Engine   active (running)
$ curl -sk https://localhost:8443/health | jq
{
  "status": "HEALTHY",
  "ruleBundleVersion": 37,
  "nativeCore": "loaded",
  "aerospike": "UP",
  "controlPlane": "REACHABLE",
  "virtualThreads": true
}
$ # smoke-test a transaction (synthetic; no real card data)
$ curl -sk https://localhost:8443/evaluate -H "Authorization: Bearer $TOKEN" \
    -d '{"amount":25000,"mcc":"5411","ip_vpn_or_proxy":true}' | jq
{ "action": "HOLD", "score": 25, "triggeredRuleIds":[100], "reasons":["VPN hold"] }
```

Green means go

`status: HEALTHY` with a non-zero `ruleBundleVersion`, native core loaded, and Aerospike UP confirms a fully wired install. Point one PSP API node at it in shadow mode first.

14 Upgrades & rollback

Method	Upgrade	Rollback
Container	`docker compose pull && docker compose up -d`	Re-pin previous image tag & `up -d`
apt	`apt update && apt install --only-upgrade hokeka-edge-engine`	`apt install hokeka-edge-engine=<old>`
dnf	`dnf upgrade hokeka-edge-engine`	`dnf downgrade hokeka-edge-engine`
Helm	`helm upgrade edge hokeka/edge-engine`	`helm rollback edge`

> **Rules vs. software**
> Rule changes are *not* upgrades — they are pulled automatically and hot-swapped with zero downtime. Software upgrades follow the table above and are safe to do one instance at a time behind your load balancer.

15 Troubleshooting

Symptom	Likely cause	Action
`nativeCore: "not loaded"`	glibc too old / lib missing	Confirm glibc ≥ 2.31; use the container image which bundles everything.
`controlPlane: UNREACHABLE`	Outbound 443 blocked / mTLS cert	Allow egress to `edge.hokeka.com`; check client cert paths. Edge keeps running on the cached bundle.
Bundle `signature FAILED`	Wrong pinned key / tampered artifact	Re-copy `hokeka-cp.pub` from onboarding; never disable verification.
All decisions `HOLD`	Fail-closed: Aerospike down or no bundle	Check Aerospike (`asadm -e info`) and that a bundle version is active. This is safe-by-design, not a silent failure.
High p99 latency	Under-provisioned / cold Aerospike	Move up a hardware tier (§3.1); put Aerospike on NVMe / a dedicated node.

```
root@psp-host: ~

# journalctl -u hokeka-edge -n 50 --no-pager     # service logs
# docker compose logs --tail=50 edge            # container logs
```

16 Prerequisite recommendations — summary

Category	Minimum	Recommended (production)
CPU	4 vCPU	8–16 vCPU, ~25% burst headroom
Memory	16 GB	32–64 GB (RAM tracks Aerospike feature volume)
Storage	100 GB SSD	250 GB–1 TB NVMe
OS	64-bit glibc ≥ 2.31 Linux, or Windows Server 2019+	Ubuntu 24.04 LTS / RHEL 9, or the container image
Java	JRE 25 (bundled)	Bundled Temurin JRE 25 — leave as shipped
Feature store	Aerospike 6.4+ co-located	Dedicated Aerospike node/cluster on NVMe
Network	Outbound 443 to Hokeka	Outbound 443 + local 8443; no inbound internet
Availability	1 instance	≥ 2 edge instances behind a load balancer
Clock	NTP enabled	chrony, < 1 s skew (token/signature validity)

> **One-line recommendation**
> For most PSPs, deploy
> **Method A (container)**
> with
> **2 instances**
> of the
> **Medium tier**

> (8 vCPU / 32 GB / NVMe) behind your load balancer, Aerospike on NVMe, outbound-443 egress to Hokeka — and scale tier/replicas as sustained TPS grows.